# INDEX

NAME: Sharweshvan STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: Computer Network - Obs

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| 1. | 20/7/26 | Network Commands | | |
| 2. | 21/7/26 | Study of Network Cables | | |
| 3. | 27/7/26 | Experiments on CISCO Packet Tracer | | |
| 4. | 28/7/26 | Configure LAN | | |
| 5. | 4/8/26 | Experiments on Packet Capture tool | | |
| 6. | 11/8/26 | Hamming Code | | |
| 7. | 24/8/26 | SLIDING WINDOW PROTOCOL | | |
| 8. | 21/8/26 | NMAP | | |
| 9. | 8/9/26 | SubNetting | | |
| 10.a) | 15/9/26 | CISCO PACKETTRACER | | |
| 10.b) | 15/9/26 | Wireless Router, DHCP server & internet cloud | | |
| 11.a) | 28/9/26 | Static Routing Configuration | | |
| 11.b) | 28/9/26 | RIP using CISCO PACKET TRACER | | |
| 12. | 29/9/26 | End To End Communication at Transport Layer | | |
| 13. | 12/10/26 | PING Program | | |
| 14. | 12/10/26 | RAW Sockets | | |

Completed 12/10/26

Aim:

To study various network commands used in Linux and Windows.

Commands:

arp -a: ARP is a short form of address resolution protocol, It will show the IP address of your computer along with the IP address and MAC address of your router.

~~hostn~~ O/P:

-gateway (192.168.1.1) at a4:2b:8c:14:55:77 [ether] on eth0

devices.local (192.168.1.15) at b8:27:eb:8a:32:11 [ether] on eth0

hostname: This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

O/P: ubuntu-server

ipconfig /all: This command displays detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP and type of Ethernet adapter in your system.

O/P:

1. lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN inet 127.0.0.1/8 scope host

2. eth0: <BROADCAST, MULTICAST, UP, LOWER_UP> mtu 1500 qdisc pfifo_fast state UP inet 192.168.10.24 brd 192.168.1.255 scope global eth0

nbtstat -a: This command helps solve problems with NetBIOS name resolution (Nbt stands for NetBios over TCP/IP)

$nmblookup WORKGROUP

192.168.1.255 WORKGROUP<00>

192.168.1.10 WORKGROUP<00>

netstat -r: Displays active TCP/IP connections, interface statistics or the system's routing table

$netstat -r

Kernel IP routing

| Destination | Gateway | Genmask | Flags | MSS | Window | IFace |
|---|---|---|---|---|---|---|
| default | gateway | 0.0.0.0 | UG | 0 | 0 | eth0 |
| 192.168.1.0 | 0.0.0.0 | 255.255.255.0 | U | 0 | 0 | eth0 |

nslookup: A tool used to perform DNS lookups on Linux. It is used to display DNS details such as IP address of particular computer, the MX records for a domain or NS servers of a domain.

$nslookup www.google.com

Server: 8.8.8.8

Address: 8.8.8.8#53

Non-Authoritative answer

Name: www.google.com

Address: 142.250.190.36

pathping: Combines ping and traceroute to trace the route to a destination and gather statistics on packet loss at every router hop.

$ mtr google.com MY traceroute [v0.92]

ubuntu-server (192.168.1.10) Packets Pings

Host Loss% Snt Last

b) Assign an IP address to an interface

[root@server ~]# sudo ip address add 192.168.1.254/24 dev enp0s3

enp0s3: <UP_BROADCAST_RUNNING>
inet 192.168.1.10/24
inet 192.168.1.254/24

c) Delete an IP address from an interface

[root@server ~]# sudo ip address del 192.168.1.254/24 dev enp0s3

No output - IP address removed successfully.

d) Bring interface online

[root@server ~]# sudo ip link set enp0s3 up

No output - interface is now UP

e) Bring interface offline

[root@server~]# sudo ip link set enp0s3 down

No output interface is now DOWN

f) Enable promiscuous mode

[root@server~]#sudo ip link set enp0s3 promiscon

No output- promiscuous mode enabled

g) Add a default route

[root@server ~]#sudo ip route add default via 192.168.1.254 dev enp0s3

No output - default route added

h) Add a route via the gateway

[root@server ~]# sudo ip route add 192.168.1.0/24 via 192.168.1.254

No output - route added successfully.

i) Delete the route

[root@server ~]# sudo ip route delete 192.168.1.0/24 via 192.168.1.254

No output - route deleted successfully.

k) Display route taken for IP 10.10.14

[root@server ~]# ip route get 10.10.1.4

10.10.1.4 via 192.168.1.254 dev enp0s3 src 192.168.1.10 cache

2) ifconfig : It is a staple in many sysadmin's tool belt for configuring and troubleshooting networks. It has since been replaced by the ip command discussed above.

3) mtr : Matt's traceroute is a program with a command line interface that serves as a network diagnostic and troubleshooting tool. It combines the functionality of the ping & traceroute commands

a Basic MTR Command (shows hops, packet loss, and response time)

```
[root@server ~]# mtr google.com
                My traceroute [v0.95]
Server (192.168.1.10)                    2026-04-08
Host                    Loss%  Snt  Last  Avg  Best  Wrst StDev
1. 192.168.1.1          0.0%   10   0.8   0.9   0.7   1.2   0.2
2. 10.10.0.1            0.0%   10   4.3   4.5   4.1   5.2   0.3
3. 142.250.66.206       0.0%   10  15.2  15.5  14.9  16.2   0.4
```

b. Show numeric IP addresses only

```
[root@server ~]# mtr -n google.com
                My traceroute [v0.95]
Host                    Loss%  Snt  Last  Avg  Best  Wrst StDev
1. 192.168.1.1          0.0%   10   0.8   0.9   0.7   1.2   0.2
2. 10.10.0.1            0.0%   10   4.3   4.5   4.1   5.2   0.3
3. 142.250.66.206       0.0%   10  15.2  15.5  14.9  16.2   0.4
```

c. Show both IP addresses and hostnames

```
[root@server ~]# mtr -b google.com
                My traceroute [v0.95]
Host                    Loss%  Snt  Last  Avg  Best  Wrst StDev
1. 192.168.1.1 (router) 0.0%   10   0.8   0.9   0.7   1.2   0.2
2. 10.10.0.1 (isp-gw)   0.0%   10   4.3   4.5   4.1   5.2   0.3
3. 142.250.66.206 (google) 0.0% 10  15.2  15.5  14.9  16.2   0.4
```

sudo tcpdump -i eth0

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

10:12:01.123456 IP 192.168.1.10.54321 > 8.8.8.8.53: UDP, length

d. Capture only 10 packets

sudo tcpdump -i eth0 -c 10

listening on eth0, link-type EN10MB

10 packets captured

12 packets received by filter

e. Capture Traffic to/from 8.8.8.8

sudo tcpdump -i eth0 -c 10 host 8.8.8.8

10:15:02 IP 192.168.1.10.45678 > 8.8.8.8.53: UDP, length 32

f. Capture Traffic Going to 8.8.8.8

sudo tcpdump -i eth0 dst host 8.8.8.8

10:16:01 IP 8.8.8.8.53 > 192.168.1.10.56789: UDP, length 48

g. Using Subnet Mask

sudo tcpdump -i eth0 net 10.1.0.0 mask 255.255.255.0

10:18:30 IP 10.1.0.5.443 > 10.1.0.20.51514: Flags [P.]

h. Using CIDR Notation

sudo tcpdump -i eth0 net 10.1.0.0/24

10:19:05 IP 10.1.0.8 > 10.1.0.25: ICMP echo request

i. Capture DNS traffic for 8.8.8.8

sudo tcpdump -i eth0 host 8.8.8.8 and port 53

10:21:11 IP 192.168.1.10.40122 > 8.8.8.8.53: UDP, length 32

10:21:11 IP 8.8.8.8.53 > 192.168.1.10.40122: UDP, length 48

5. Ping: It is a tool that verifies IP-level connectivity to another TCP/IP computer by sending Internet Control Message Protocol (ICMP) Echo Request messages.

```
[root@server ~]# ping google.com
PING google.com (216.58.206.174) 56(84) bytes of data.
64 bytes from sof02s27-in-f14.1e100.net (216.58.206.
174): icmp_seq=1 ttl=56 time=10.7 ms
64 bytes from sof02s27-in-f14.1e100.net (216.58.206.
174): icmp_seq=1 ttl=56 time=10.2 ms.
```

Stop ping command by "CTRL+C".

1. Which command is used to find reachability of a host machine from your device?

   ping <hostname_or_IP>

2. Which command gives details of hops taken by a packet to reach its destination?

   traceroute <hostname_or_IP>

3. Which command displays the IP configuration of your machine?

   ip addr

4. Which command displays the TCP port status on your machine?

   ss -t

5. Write the command to modify IP configuration on a Linux machine.

   sudo ip addr add 192.168.1.100/24 dev enp0s3
   sudo ip addr del 192.168.100/24 dev enp0s3
   sudo ip link set enp0s3 up

Result:

Thus the linux commands are successfully executed and verified.

28/7/26

Exp2

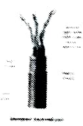
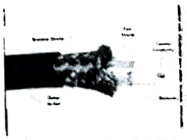
# Study Of Different Types Of Network Cables

Aim:

Study of different types of network cables.

Different types of cables used in networking:

1. Unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial Cable
4. Fibre Optic Cable.

| Cable Type | Category | Maximum Data Transmission | Advantages / Disadvantages | Application / Use | |
|---|---|---|---|---|---|
| UTP | Category -3 | 10 bps | Advantages<br>* Cheaper in cost<br>* Easy to install as they have a smaller overall diameter.<br>Disadvantages<br>* More prone to electromagnetic interference and noise. | 10 Base T ethernet<br>Fastethernet, Gigabit ethernet |  |
| | Category -5 | Up to 100mbps | | | |
| | Category 5e | 1 Gbps | | | |
| STP | Category 6, 6a | 10Gbps | Advantages:<br>• Shielded<br>• Faster than UTP<br>• Less susceptible to noise and interference<br>Disadvantages<br>• Expensive<br>• Greater installation | Gigabit Ethernet, 10G Ethernet (55m)<br>Widely used in data centres<br>Gigabit Ethernet, 10G Ethernet (100m) |  |
| SSTP | Category 7 | 10Gbps | | |  |
| Coaxial cable | RG-6<br>RG-59<br>RG-11 | 10-100Mbps | • High Bandwidth<br>• Immune to interference<br>• Low loss bandwidth<br>• Versatile<br>Disadvantages<br>• Limited distance<br>• Cost<br>• Size & Bulky | ~~Speed of~~ Signal is 500m.<br>Television network<br>High speed internet connections. |  |

fibre optics cable

single mode
Multi mode

two types

- High speed
- High bandwidth
- High security
- Long distance

Disadvantage

- Expensive
- Requires skill

Maximum distance of fibre optics cable is around 100km

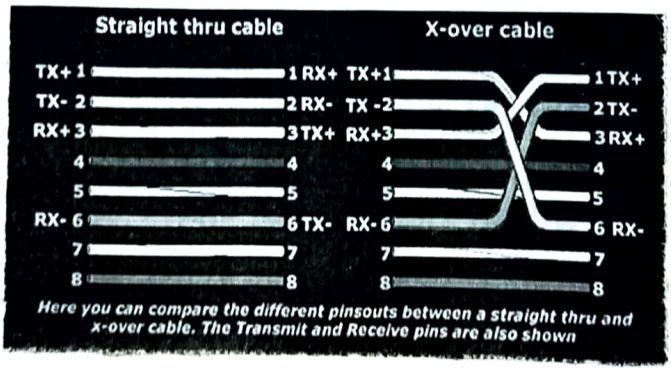


b) M.K. Your Own Ethernet Cross Over Cable

Tools and parts needed:

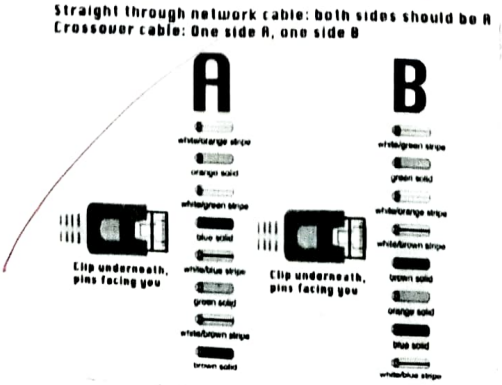
- Ethernet cabling CAT 5e is created for gigabit support, but CAT 5 cabling works as well, just over shorter distances.
- A crimping tool, This is an all in one networking tool shaped to push down the pins in the plug and strip and cut the shielding of cables.
- Two RJ45 Plugs.

Step1: To start construction of the device, begin by threading shields onto the cable.

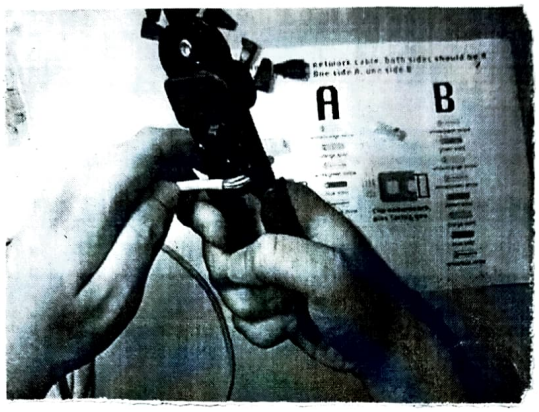


Here you can compare the different pinsouts between a straight thru and x-over cable. The Transmit and Receive pins are also shown

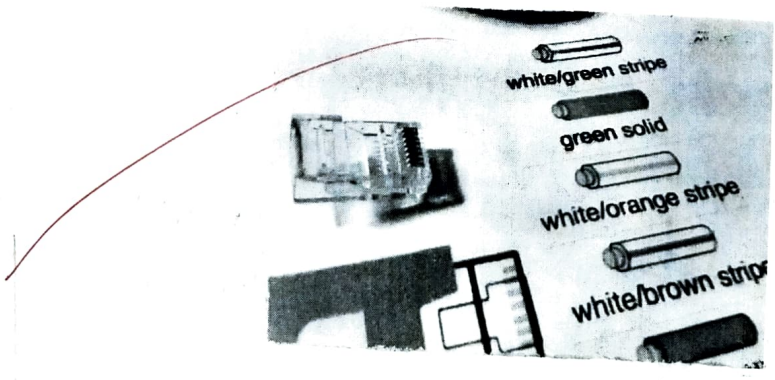
Step2: Next, strip approximately 1.5cm of the cable shielding from both ends. The crimping tool has a round area to complete task.



Step 3: After, you will need to untangle the wires, there should be four "twisted pairs". Referencing back to sheet, arrange them from top to bottom. One end should be in arrangement A and other in B.




Step 4: Once the order is correct, bunch them together in a line, and if there are any that stick out farther than others, strip them back to create an even level. The difficult aspect is placing these into RJ45 Plug without messing up the order.



3. Which type cable is used to connect a router/switch to your PC?

PC to switch: Straight cable

PC to router: Cross cable

4. Find out the category of twisted pair cable used in your lab to connect PC to network.

Most commonly used category is Cat 5e or Cat 6.

5. Write down your understanding, challenges faced and output received while making it a twisted pair cross/straight cable.

Understanding: Making an RJ45 cable requires stripping the outer insulation sheath, untwisting and arranging the inner wires in the color code.

Challenges faced:

i) Straightening the fine inner wire

ii) Ensuring all wires are cut evenly.

iii) Stripping outer jacket.

Output Received:

All Led indicator lights light up sequentially on both ends for a straight through cable.

Results

Thus the experiment has been successfully executed.

Exp 1 Packet Tracer Tool Installation and User Interface Overview

Aim: To study the packet tracer tool installation and user interface overview.

Introduction:

A simulator, as name suggests simulates network devices and its environment

1. Allows you to model complex systems.
2. Available for both Linux & Windows
3. Protocols in PT are coded to work & behave in same way as they would on real hard

Installing Packet Tracer:

Go to https://www.netacad.com then log in.

Click on packet tracer graphic and download the package appropriate for your OS.

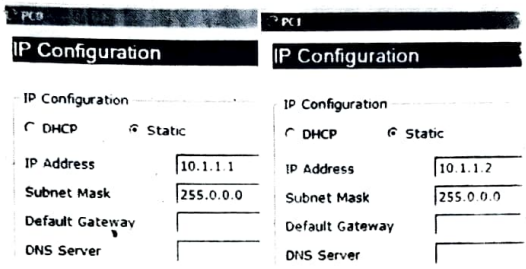
Windows: Installing in windows is pretty simp

The setup comes in a single file named Packettracer_Setup6.0.1exe.

Linux: Linux users with an Ubuntu/Debian distribution should download file for Ubuntu, and those using Fedora/Redhat/CentOS must download file for Fedora.

User Interface Overview

1. Menubar - This is a common menu found in all software applications; Used to open, save, print, change preferences and so on.
2. Main toolbar - This bar provides shortcut icons to accessed, such as open, zoom, undo and redo on right hand side is an icon for entering network information.




4. Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of simulation.

5. Repeat step #3 to step #5 for PCs connected to switch.

6. Observe the HUB and switch are forwarding the PDU and write your observation and conclusion about the behaviour of switch and HUB.

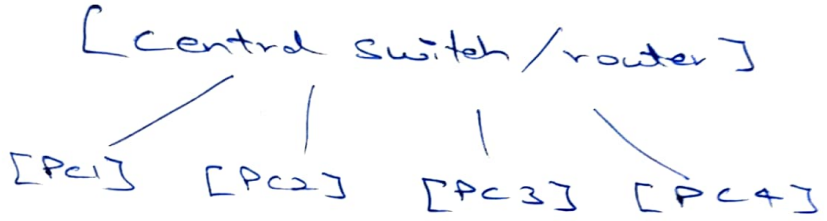
Student Observation:

a. From your observation write down the behaviour of switch and hub in terms of forwarding the packets received by them.

Switch: Reads the target MAC address and forwards data only to specific port connected to receiving device.

hub: Broadcasts incoming data to all connected ports regardless of destination.

b. Find out the network topology implemented in your college and label that topology in your observation book.




Results

The experiment has been successfully executed.

Configure a LAN (Local area network) using a switch and ethernet cables in your lab.

Aim: Setup and configure a LAN (Local area network) using a switch and ethernet cables in your lab.

What is a LAN?

A Local Area Network (LAN) refers to a network that connects devices within a limited area, such as an office building, school or home. It enables users to share resources, including data, printers and internet access.

How to setup a LAN

Step 1: Plan and design an appropriate network topology taking into account network.

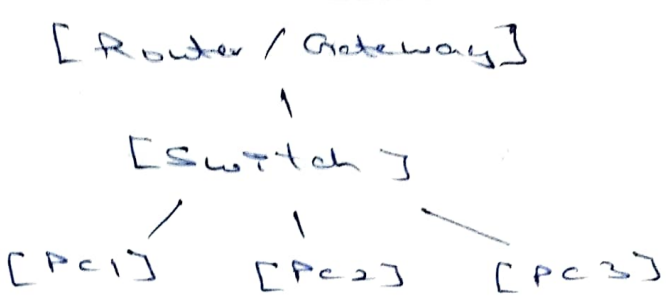
Step 2: Take 4 Computers, a switch with 8, 16 or 24 ports which is sufficient for networks of these sizes and 4 ethernet cables.

Step 3: Connect your computers to network switch via ethernet cable which is simple as plugging one end of cable in to your computer and other end into your network switch.

Step 4: Assign IP addresses to your PCs.

Step 5: Configure your network switch.

1. Connect computer to switch
2. Log in to web interface
3. Configure basic settings
4. Assign IP address as: 10.1.1.5




| Device | IP address | Subnet Mask | Default Gateway |
|---|---|---|---|
| Router (Gateway) | 192.168.1.1 | 255.255.255.0 | N/A |
| PC 1 | 192.168.1.10 | 255.255.255.0 | 192.168.1.1 |
| PC 2 | 192.168.1.11 | 255.255.255.0 | 192.168.1.1 |
| PC 3 | 192.168.1.12 | 255.255.255.0 | 192.168.1.1 |

- Outcome: Successful peer to peer communication across the local network

Result:

A LAN has been configured using a switch and ethernet cables.

Exp 5 Experiments On Packet Capture ... Wireshark

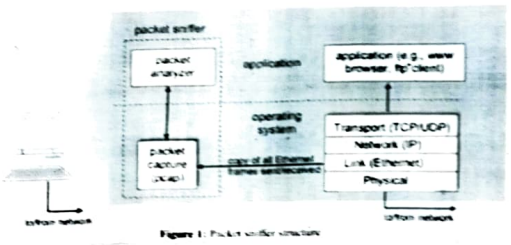
Aim: Experiments on Packet capture tool.

Packet Sniffer:

- Sniffs messages being sent/received from/by the computer
- Store and display the contents of various protocols fields in messages.
- Passive program
  - never sends packet itself
  - no packets addressed to it
  - receives a copy of all packets

Packet Sniffer Structure Diagnostic Tools

- Tcpdump - Eg: tcpdump -enx host 10.129.41.2
- Wireshark - wireshark -r exe3.out




Description

Wireshark

A network analysis tool formerly known as Ethereal, captures packets in real time and display them in human readable format.

Wireshark includes filters color coding and other features let you dig deep into network traffic

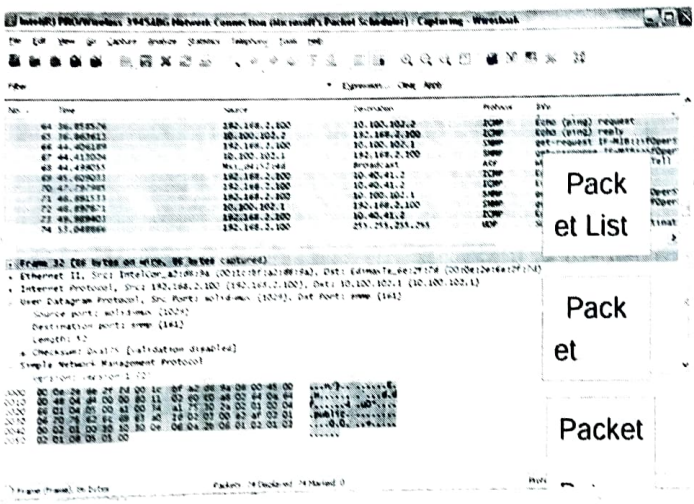
and individual packets.

What we can do with Wireshark:

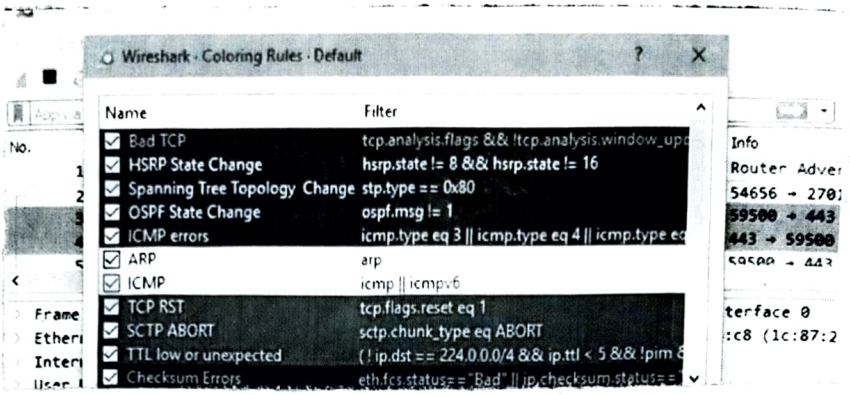
- Capture network traffic
- Decode packet protocols using dissectors
- Define filters - capture and display
- Watch smart statistics
- Analyze problems.

## Capturing Packets

After downloading and installing Wireshark, launch it and double click the name of a network interface under capture to start capturing packets on that interface.




As soon as you click on interface's name, you'll see the packets starts to appear in real time. Wireshark captures each packet sent to or from your system.




## The "Packet List" Pane

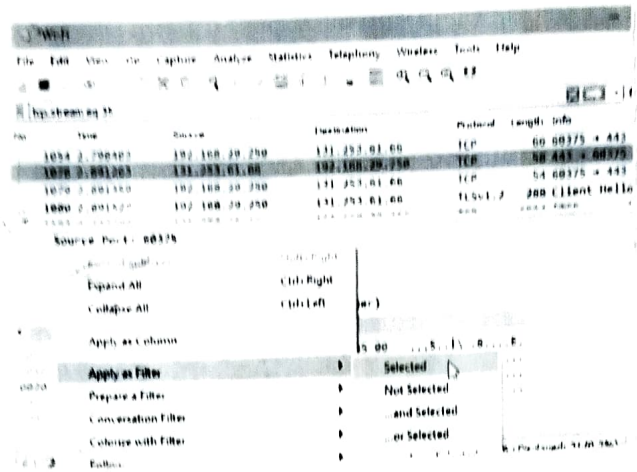
It displays all the packets in the current capture file. The "Packet List" pane each line in the packet list corresponds to one packet in capture file.

## The "Packet Details" Pane

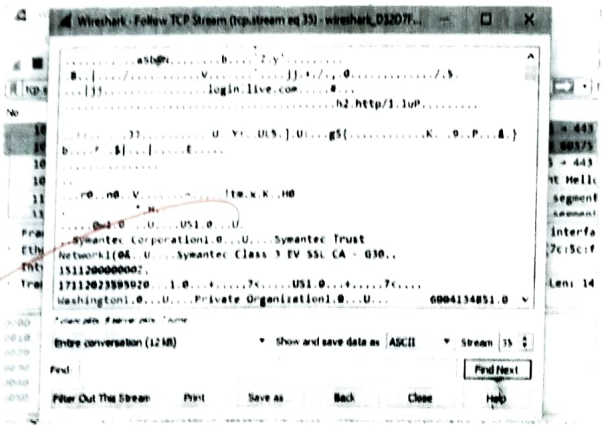
The packet details pane shows the current packet in a more detailed form. This pane shows the protocol fields of the packet selected in the "Packet List" pane.

## The "Packet Bytes" Pane

Shows the data of the current packet in a hexdump style.




Close the window and you'll find a filter has been applied automatically. Wireshark is showing you the packets that make up the conversation.

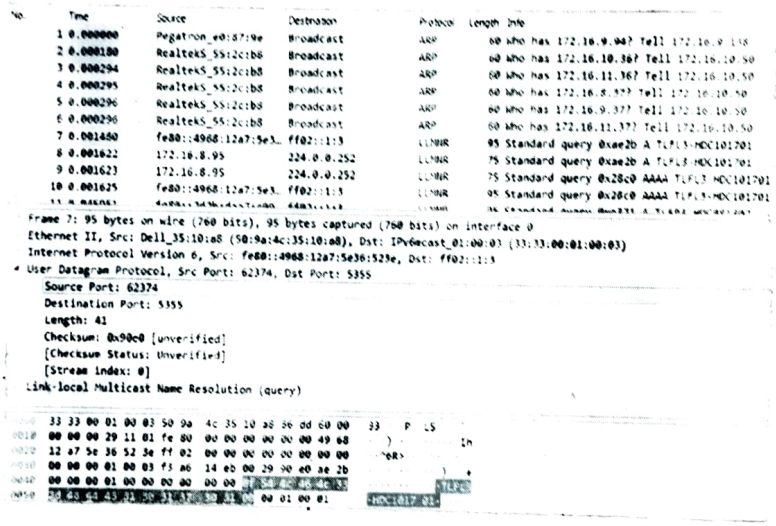


# Capturing and Analyzing Packets Using Wireshark Tool

To filter, capture, view, packets in Wireshark Tool Capture 100 packets from the Ethernet: IEEE 802 LAN Interface and save it

Procedure:

- Select Local Area Connection in Wireshark.
- Go to capture option.
- Select stop capture automatically after 1000 pack
- Then click start capture.
- Save the packets.




1. Create a filter to display only TCP/UDP packets, inspect the packets and provide

→ Select Local Area Connection.

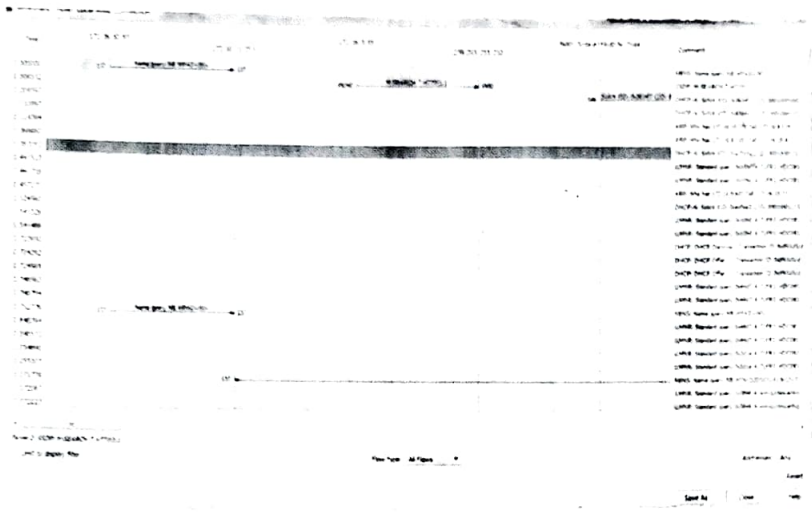
→ Go to capture option.

→ Select stop capture automatically

→ Click Start capture

→ Search TCP packets

→ Save packets.



2. Create a filter to display only ARP packets & inspect packets.

→ Go to capture option

→ Select stop capture automatically

→ Then click start capture

→ Save packets.

3. Create a filter to display only DNS packets and provide flow graph

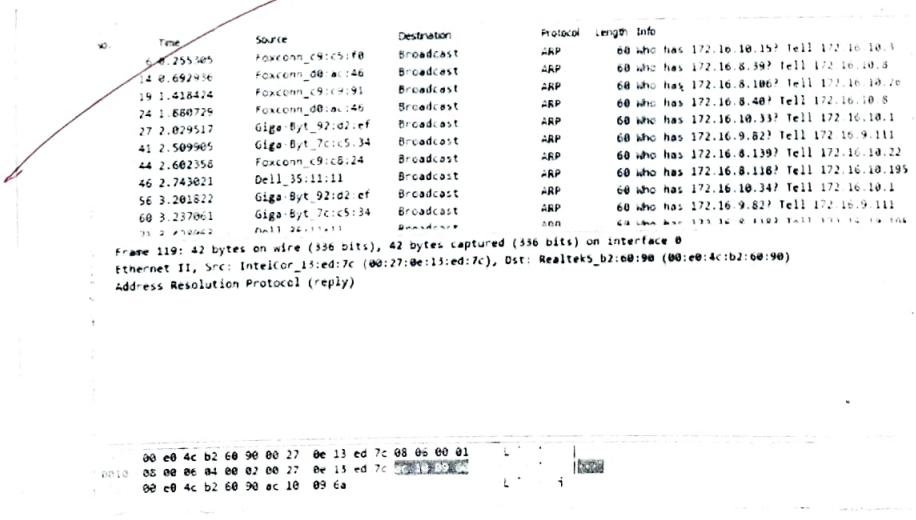
→ Go to capture option.

→ Select stop capture automatically

→ Then click start capture

→ Search DNS packets

→ Click Statistics.



2. How ARP protocol is different from other handled?

ARP operates directly on top of the Data link layer and encapsulates directly in ethernet frames.

3. Which transport layer protocol is used by DNS?

DNS uses UDP for standard queries and TCP for zone transfers.

4. Port number used by http protocol?

Port 80.

5. What is broadcast ip address?

A special IP address used to send data simultaneously to all hosts within a given network or subnet.

Result:

The experiment has been successfully executed.

Exp6

# Hamming Code Concept

Aim: Write a program to implement error detection and correction using HAMMING code concept. Make a run test to input data stream and verify error correction feature.

## Error Correction at Data Link Layer:

It is a set of error correction codes that can be used to detect and correct the errors that can occur when data is transmitted from the sender to receiver.

## Create Sender Program

1. Input to sender file should be a text of any length.
2. Apply hamming code to add redundant bits
3. Save output in a file called channel.

## Create receiver program

1. Should read input from channel file.
2. Apply hamming code on binary data.
3. If error, display the position.
4. Else remove the redundant bits and convert the binary data to ascii and display the output.

Observation Code:

```
def calculate_parity_bits(data_length):
    r=0
    while (2** r)<(data_length+r+1):
        r+=1;
    return r

def encode_hamming(code):
    m = len(data)
    r = calculate_parity_bits(m)
    total = m+r
    code=['0']
    i=0
    for i in range (1, total+1):
        if(i & (i-1))! =0:
            code[i] = data[j]
            j+=1;
    for p in range(r)
        pos = 2** p
        parity =0
        for i in range (1, total+1):
            if i & pos:
    code [pos] str(parity)
    return ''.join(code : ])
```

Output:

Original text: HELLO

Binary data: 0100100001000100101100110001

Hamming code: 0100100001000100101100110001...

data saved successfully in result.txt

If error:

Error detected at position: 25

Error corrected successfully.

Recovered data: Hello

Exp 7 Control Flow At Data Link Layer Using

# SLIDING WINDOW PROTOCOL

Aim: Write a program to implement flow control at data link layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one node to another.

Create a sender program with following features:-

1. Input window from user.
2. Input text message from user.
3. Consider 1 character per frame.
4. Create a frame with [Frame no, DATA].
5. Print output on screen and save it in a file called Sender_Buffer.
6. Wait for acknowledgment from receiver.
7. Reader a file called Receiver_Buffer.
8. Check ACK field for Acknowledgement numbe
9. If Acknowledgement number is accepected, se new set of frames accordingly.

Create a receiver file with following features:-

1. Reader a file called Sender_Buffer.
2. Check Frame no.
3. If frame no. are as expected, write the appropriate ACK no. in Receiver_Buffer file. Else write NACK no. in the Receiver_Buffer

## Sender Program:

```
import time
import os
sender_buffer = "sender_buffer.txt"
receiver_buffer = "Receiver_buffer.txt"
w_s = int(input("Enter size:"))
msg = input("Enter text:")
frames = []
for i in enumerate[msg]:
frames.append((i,ch))
n_f = 0
e_A = 0
```

## Receiver Program:

```
import time
import os
sender_Buffer = "sender_buffer.txt"
receiver_Buffer = "Receiver_buffer.txt"
print("====Receiver=====")
print("Waiting for frames...\n")
with open(Receiver_Buffer, "w")
as file:
    file.write(f"Ack = {expected from"}
    break.
```

## Output

### Sender:

```
Message: HELLO
Total frames: 5
Sending Frame 0: Data = H
Sending Frame 1: Data = E
Sending Frame 2: Data = L
Waiting for ACK
```

### Receiver:

```
Frame 0 : Data = H
Frame 0 accepted
Frame 1 : Data = E
Frame 1 accepted

All frames transmitted
successfully.
```

Exp-8 Discover Live Host Using Nmap Scan

a) (ARP, ICMP, TCP/UDP) on the Try Hack Me Platform

Aim: To discover live hosts using Nmap scans (ARP, ICMP, TCP/UDP) on the TryHackMe Platform Room

Link: https://tryhackme.com/room/nmap01

Introduction:

When targeting a network we need an efficient tool to handle repetitive task. This tool should help us find out which systems are active and what services are running on systems. The tool that we will rely on is Nmap. The room is the first in series of four rooms dedicated to Nmap. The room is the first of four in this Nmap series.

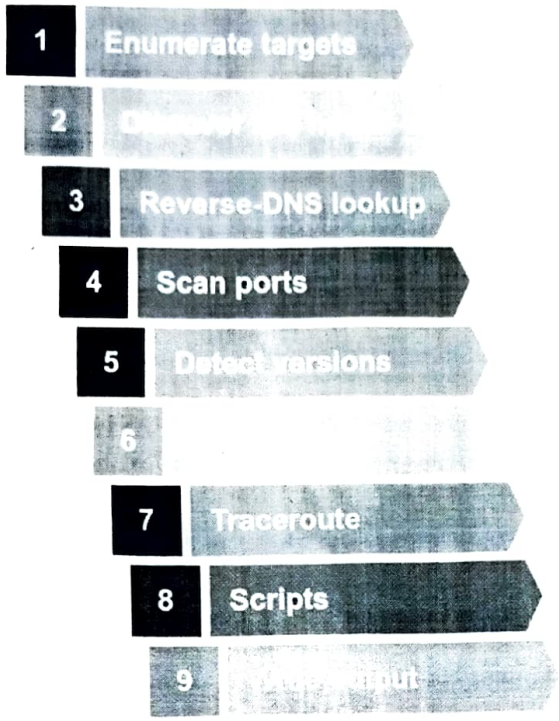
- Nmap Live Host Discovery
- Nmap Basic Port Scans
- Nmap Advanced Port Scans
- Nmap Post Port Scans

This room explains the steps that Nmap carries out to discover the systems that are online before port-scanning. This stage is crucial because trying to portscan offline systems will only waste time and create unnecessary noise on network.

ARP scan: uses ARP requests to discover live hosts.

ICMP scan: uses ICMP requests to identify live hosts.

TCP/UDP ping scan: sends packets to TCP ports and UDP ports to determine live hosts.




## Subnetworks

A network segment is a group of computers linked through a shared medium, like an Ethernet switch or Wi-Fi access point. In IP networks, a subnetwork typically consists of one or more network segments connected together and configured to use the same router. A network segment is a physical connection while a subnetwork is a logical connection.

Exp-8

b)

# Wireless LAN using Cisco Packet Tracer

Aim: Configuration of Wireless LAN using Cisco Packet Tracer.

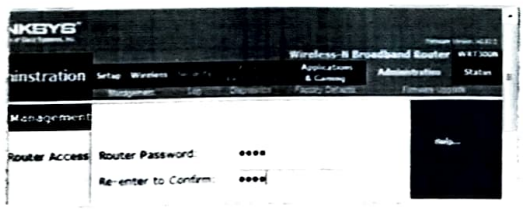
Design a topology with three PCs connected from Linksys Wireless routers.

Perform the configuration:

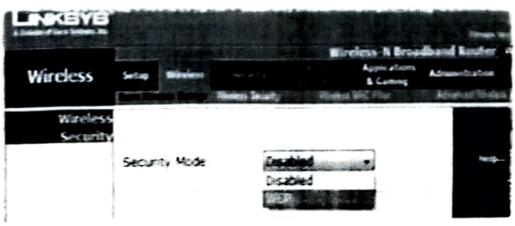
- Configure static IP on PC and Router.
- Set SSID to mothernetwork
- Set IP address of router to 192.168.0.1, PC0 to 192.168.0.2, PC1 to 192.168.0.3 and PC2 to 192.168.0.4.
- Connet PC using WAP key.

Step 1:- Click on wireless router

- Select administration tab from top menu, set username and password to admin and save settings.




- Click on wireless tab and set default SSID to mother network.



It will connect you with wireless router.
System is connected. PC1 card is active
Repeat same process on PC1 and PC2.




Student Obs:

1. What is SSID of wireless router?
SSID (Service Set Identifier) is the unique name assigned to a wireless router that identifies its WiFi network to nearby devices.

2. What is security key in wireless router?
A security key is confidential password required to connect to a secure wireless network.

Result:
The experiment has been successfully executed.

Exp 9

Aim: Implementation of SUBNETTING in CISCO PACKET TRACER Simulator.

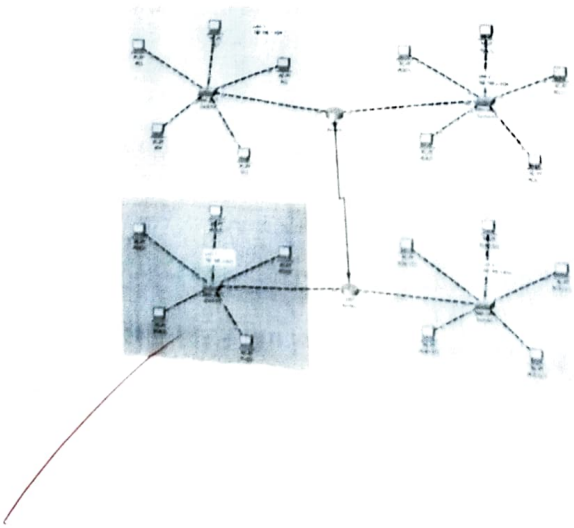
Classless IP subnetting is a technique that allows for more efficient use of IP addresses by allowing for subnet masks that are not just the default masks for each IP class.

CREATING A NETWORK TOPOLOGY:

The first step in implementing classless IP subnetting is to create a network topology in Packet Tracer. To create a network topology in Packet Tracer, select the "New" button in top left corner, then select "Network" and "Generic".

ADDING THE DEVICES:

Once we have created our network topology, we can add devices to it. Here, we will be adding routers, switches and PCs.



The IP addressing for network follows:

- Router R1:
- Gigabit Ethernet 0/0: 192.168.1.1
- Gigabit Ethernet 0/1: 192.168.2.1
- Switch S1:
- Fast Ethernet 0/1: 192.168.1.0/27
- PC1: 192.168.1.11
- PC2: 192.168.1.12
- PC3: 192.168.1.13
- PC4: 192.168.1.14
- PC5: 192.168.1.15
- Fast Ethernet 0/2: 192.168.2.0/27
- PC1: 192.168.2.11
- PC2: 192.168.2.12
- PC3: 192.168.2.13
- PC4: 192.168.2.14
- PC5: 192.168.2.15
- Router R2:
- Fast Ethernet 0/0: 192.168.3.1
- Fast Ethernet 0/1: 192.168.4.1
- Switch S2:
- Fast Ethernet 0/1: 192.168.3.0/27
- PC1: 192.168.3.11
- PC2: 192.168.3.12
- PC3: 192.168.3.13
- PC4: 192.168.3.14

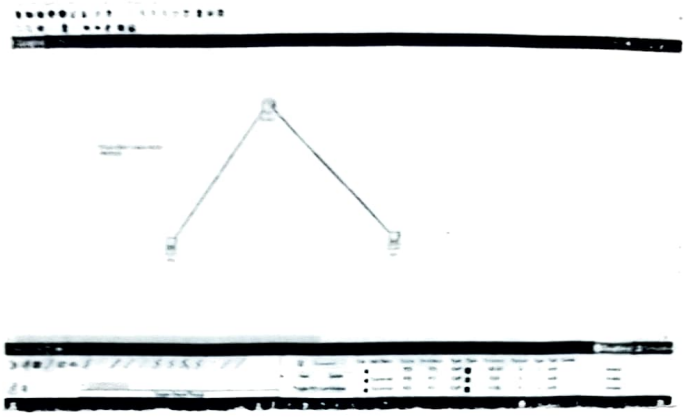
Result:

The experiment has been successfully executed.

Exp 3:

Aim: Inter-working with routers in CISCO PACKET TRACER simulator.

a) In network, a router and 2 PCs are used. Computers are connected with routers using a copper straight-through cable.



Procedure:

Step 1:

1. Select the router and Open CLI
2. Press ENTER to start configuring Router1
3. Type enable to activate privileged mode

Step 2:

1. Assign IP address to every PC in network
2. Select PC, Go to desktop and select IP configuration and assign an IP address
3. Assign the default gateway of PC0 as 192.168

Step 3:

1. Connect FastEthernet0 port of PC0 with FastEthernet 0/0 port of Router1 using a copper straight through cable.

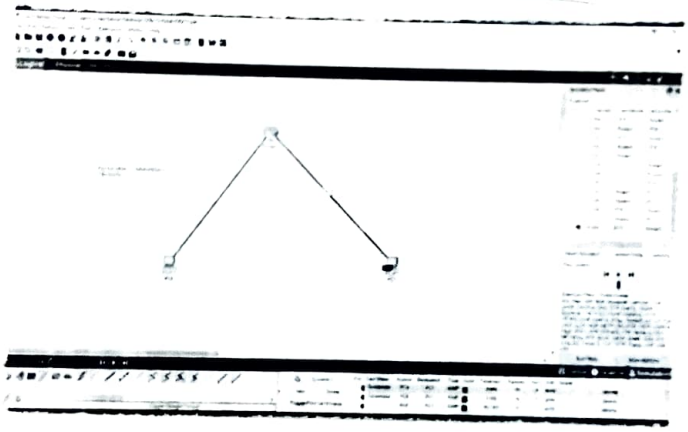
Router Configuration Table:

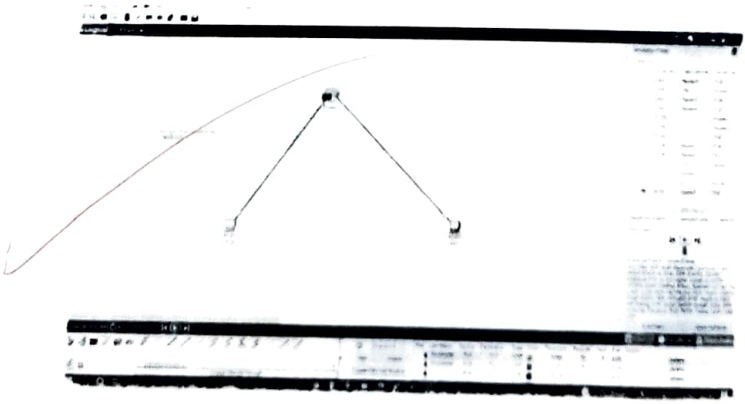
| Device Name | IP address Fast Ethernet | Subnet Mask | IP address Port Number |
|---|---|---|---|
| Router1 | 192.168.10.1 | 255.255.255.0 | 192.168.20.1 |

PC Configuration Table

| Device Name | IP address | SubnetMask | Gateway |
|---|---|---|---|
| PC0 | 192.168.10.2 | 255.255.255.0 | 192.168.10.1 |
| PC1 | 192.168.20.2 | 255.255.255.0 | 192.168.20.1 |

Sending a PDU From PC0 to PC1:

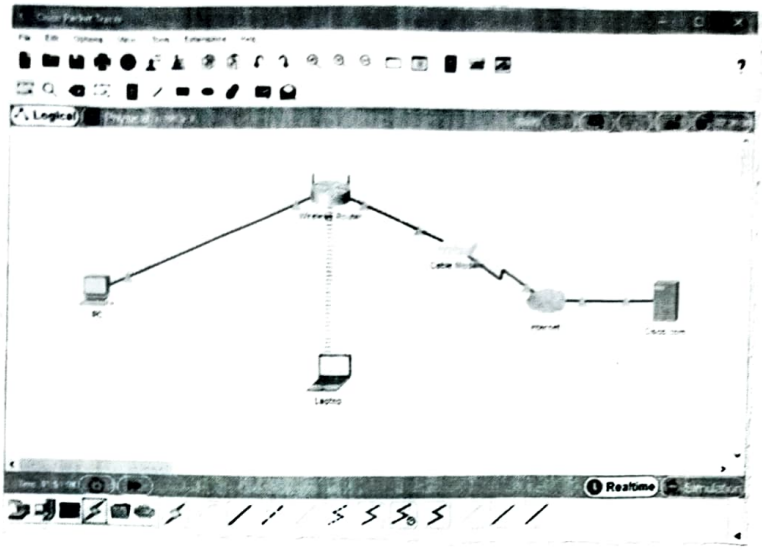




Result:

The ~~code~~ experiment was successfully executed.

1 b) Aim: Design and configure an internetwork using wireless router, DHCP server and

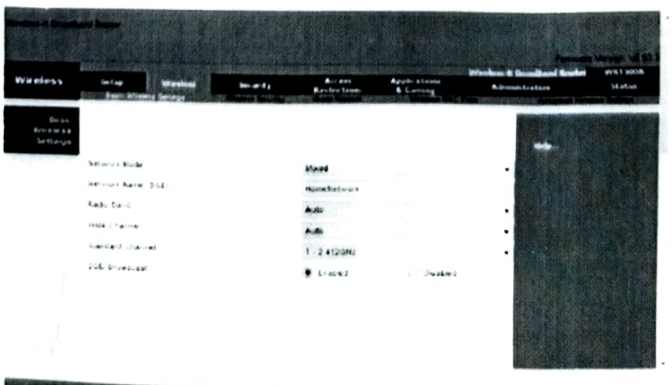


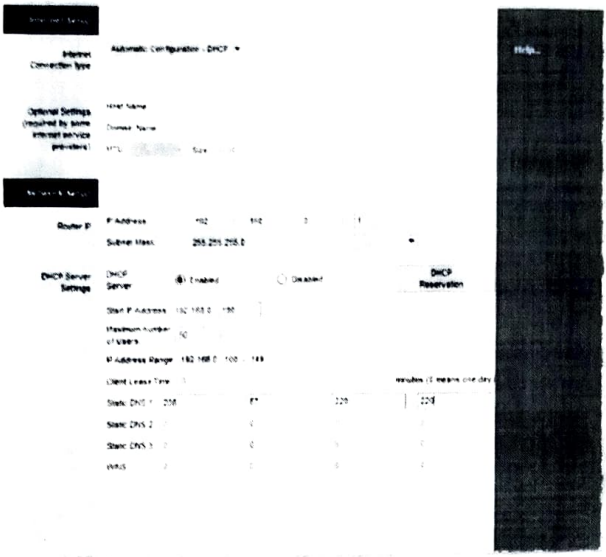

Addressing Table

| Device | Interface | IP address | Subnet Mask | Default Gateway |
|---|---|---|---|---|
| PC | Ethernet() | DHCP | | 192.168.0.1 |
| Wireless Router | LAN | 192.168.0.1 | 255.255.255.0 | |
| Wireless Router | Internet | DHCP | | |
| Cisco.com Server | Ethernet() | 209.67.220.226 | 255.255.255.0 | |
| Laptop | Wireless() | DHCP | | |

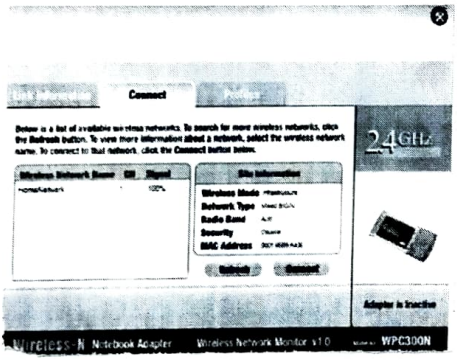
Step 1:

Create wireless network on wireless router





Step 2: Configure laptop to access the wireless network. Click on laptop icon on Packet Tracer Logical workspace and on laptop configuration windows select Physical tab.




Step 3: Configure the PC. Click on PC icon on Packet Tracer Logical workspace and select the Desktop tab and then IP Configuration.

Step 4:

a. Install network modules if necessary.

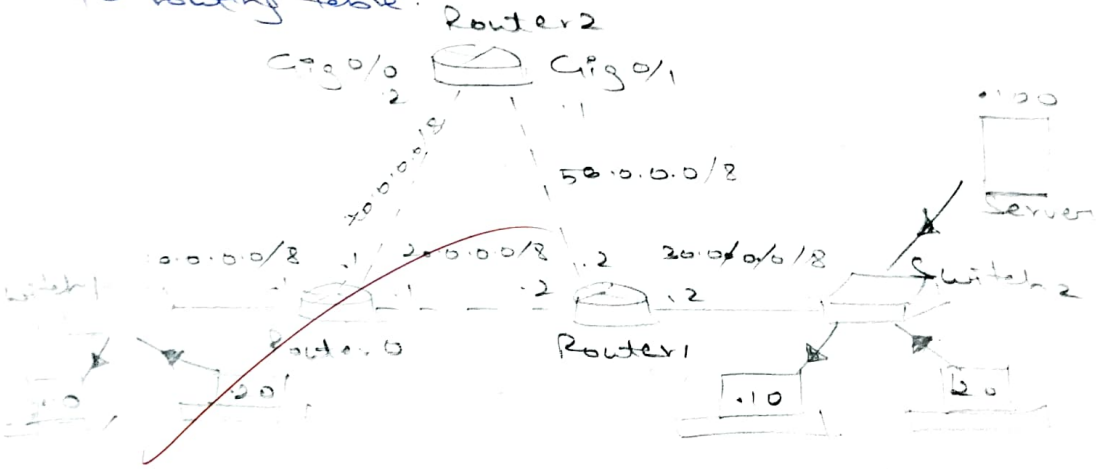
b. Identify From and To Ports.

c. Identify type of provider.

Exp 11

a) **Aim:** Simulate Static Routing Configuration using Cisco Packet Tracer.

Static routes are the routes you manually add to the router's routing table. The process of adding static routes to routing table is known as static routing. Apcket tracer to understand how to use static routing to create and add a static route to routing table.




The table lists the connected network

| Router | Available networks on local interfaces | Networks avail on other interfaces |
|---|---|---|
| Router 0 | 10.0.0.0/8, 20.0.0.0/8, 40.0.0.0/8 | 30.0.0.0/8, 50.0.0.0/8 |
| Router 1 | 20.0.0.0/8, 30.0.0.0/8, 50.0.00/8 | 10.0.0.0/8, 40.0.0.0/8 |
| Router 2 | 40.0.0.0/8, 50.0.0.0/8 | 10.0.0.0/8, 20.0.0.0/8, 30.0.0.0/8 |

**Result:**

The experiment has been successfully executed.

Exp 11

b) Aim: Simulate PIP using CISCO Packet Tra

Initial IP configuration:

| Device | Interface | IP Configuration | Connected |
|---|---|---|---|
| PC0 | FastEthernet | 10.0.0.2/8 | Router's Fa |
| Router0 | Fa0/1 | 10.0.0.1/8 | PC's Fast Et |
| Router0 | S0/0/1 | 192.168.1.254/30 | Router2's S |
| Router0 | S0/0/0 | 192.168.1.249/30 | Router1's S0/ |
| Router1 | S0/0/1 | 192.168.1.253/30 | Router1's S0 |
| Router1 | S0/0/1 | 192.168.1.245/30 | Router0's S0 |
| Router2 | S0/0/1 | 192.168.1.253/30 | Router 0's S0 |
| Router2 | Fa/0/1 | 20.0.0.1/30 | PC 1's Fast |
| PC1 | FastEthernet | 20.0.0.2/30 | Router2's Fa |

Assign IP address to PCs

Assign IP address to interface of Routers

Double click Router0 and click CLI and press Enter key to access the command prompt of Router0.

Following commands are used to access global configuration mode.

Router> enable

Router# configure terminal

Enter configuration commands, one per line.

End with CNTL/Z.

Router1

Router> enable

Router# configure

terminal

Enter configuration commands, one per line. End with CNTL/Z.

Router(config)#interface serial 0/0/0.

Router(config)# no shutdown

Router(config)# exit

Router(config)# interface serial 0/0/1

Router(config)# clock rate 64000

Router(config)# bandwidth

Shutdown Router config

Router2:

Router >enable

Router# configure

terminal

Enter configuration commands, one per line. End with CNTL/Z.

Router(config)# interface fastEthernet 0/0.

Router(config)#interface ~~ip~~address 20.0.0.1

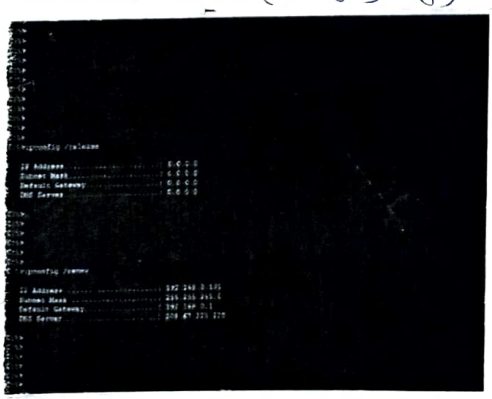
255.0.0.0 Router(config-if)# no shutdown

Router(config-if)#ip address 192.168.1.245

Router(config)# exit

Router(config-if)#ip address 192.168.1.253

255.255.255.252 Router(config-if)# no shutdown.



Result:

The ~~practical~~ experiment has been successfully executed.

Exp 12 End-End Communication at Transport Layer

a) Implement echo client/server using TCP/UDP sockets

TCP

```
import socket
import threading

# --- Server ---
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen()
    print("[SERVER] Listening on 127.0.0.1:5000")
    conn, addr = server.accept()
    print(f"[SERVER] Connected by {addr}")

# --- CLIENT ---
def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] Connected to server")
    while TRUE:
        msg = input("Enter message (or 'quit' to exit): ")
        if msg.lower() == 'quit':
            break
        print(f"[SERVER RESPONSE] {response}")
    client.close()

# --- MAIN ---
if __name__ == "__main__":
    # Run server in background
    server_thread = threading.Thread(target=start_server,
                                     server, daemon=True)
    start_client()
```

Output:

```
= = = = RESTART: C:/Users/tkela/Downloads/
[SERVER] Listening on 127.0.0.1:5000 [CLIENT]
[SERVER] Connected by ('127.0.0.1', 53488) Enter message
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
= = = = RESTART: C:/Users/tkela/Downloads/happy = = = =
[SERVER] Listening on 127.0.0.1:5000 [CLIENT]
[SERVER] Connected by ('127.0.0.1', 60328) Enter message
HI
[SERVER] Received: HI
[SERVER RESPONSE] Server Received: HI
```

UDP

```
import socket
import threading
# ---SERVER----
def udp_server():
    s = socket.socket(socket.AF_INET, socket.
                      SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")

# ---CLIENT----
def udp_client():
    c = socket.socket(socket.AF_INET, socket.
                      SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit' to stop):")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] Closing connection...")
            c.close()
```

Output:

```
Enter message (or 'quit' to stop): [SERVER]
        Running on 127.0.0.1 :12000 hi
[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hel
Echo from Sever: hello
Enter message (or 'quit' to stop): 'how are yo
[SERVER] Received from ('127.0.0.1, 49670) :
                                    how are yo
Enter message (or 'quit' to stop): quit
[CLIENT] Stopped. [SERVER] Stopped.
```

Result:

The experiment has been successfully executed.

Exp13: Aim: Implement your own ping program.

```
import os
import platform
import subprocess
def ping(host="google.com", count=4):
    param="-n" if platform.system().lower()=="windows"
                                    else "-c"
    command=["ping", param, str(count), host]
    try:
      print(output)
    except Exception as e:
      print(f"Ping failed: {e}")
if __name__=="__main__":
    ping("8.8.8.8")
```

Output:

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119

Ping statistics for 8.8.8.8:
    Packets: Sent=4, Received=4 Lost=0 (0% Loss)
Approximate round trip times in milli-seconds:
    Minimum=5ms, Maximum=9ms Average=7ms
```

Result:

The program has been successfully executed and verified.

Exp 14 Aim: Write a code using raw sockets to implement packet sniffing

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP

def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst
    # Determine the protocol
    protocol_name = ""
    if protocol == 1:
        protocol_name = "ICMP"
    elif protocol == 6:
        protocol_name = "TCP"
    elif protocol == 17:
        protocol_name = "Unknown Protocol"
    print(f"Protocol: {protocol_name}")
    print(f"Source IP: {src_ip}")
    print(f"Destination IP: {dst_ip}")
    print("-" * 50)
```

Output:

```
Protocol: TCP
Source IP: 57.155.141.17
Destination IP: 192.168.1.6
--------------------
Protocol: TCP
Source IP: 192.168.1.6
Destination IP: 57.155.141.17
--------------------
Protocol: TCP
Source IP: 192.168.1.6
Destination IP: 57.155.141.17
```

Protocol : TCP

Source IP 192.168.1.6

Destination IP 52.149.14.7

---

Protocol TCP

Source IP: 192.168.1.6

Destination IP 52.149.14.7

---

Protocol UDP

Source IP 192.168.1.6

Destination IP 192.168.1.1

---

Protocol: UDP

Source IP: 192.168.1.6

Destination IP: 192.168.1.1

Result

The code was successfully executed and verified.

(compiled & verified)